avanade

Do what matters

Bolt Framework

AI-Driven Development Lifecycle · Avanade

© 2026 Avanade Inc. All Rights Reserved

What Bolt Framework is

An AI-native development methodology — not a tool, not a prompt library

> *"A modular, practice-based AI-native development methodology … with intelligent micro-iterations called **Bolts**, orchestrated by specialized AI agents."*



Bolts: days, not weeks

A Bolt is a **2–3 day micro-iteration** ending in working, tested code; 4–6 per feature. That pace needs solid CI/CD — without it, we run Bolts inside very short Sprints.



A Constitution that is law

19 articles fixing stack, architecture, quality and security. Agents must read it, validate against it and **fail** on violation.



A staffed agent team

51 specialised agents and 114 skills covering all six lifecycle phases — from RFP analysis to decommissioning — plus four that run across every phase: **security, dependencies, docs, templates**.



Dual-client by design

One methodology definition, two clients: **GitHub Copilot and Claude Code** read the same artifacts. No per-tool rewrite.

The deliverable is not only working software — it is an **operating model the client's own team can keep running**: the same artifacts, the same gates, in their repository, with their tooling.

Counts verified on disk at framework `v1.0.0`. Bolt Framework credits **AWS AI-DLC**, Domain-Driven Design and Clean Architecture as antecedents.

avanade

© 2026 Avanade Inc. All rights reserved.

Do what matters

The lifecycle — six gated phases

Each phase produces **one named artifact** — that is what makes it auditable

1 · INCEPTION
Define the project DNA.
`constitution.md`

2 · DISCOVERY
Turn intent into specs.
`specs/…/feature.md`

3 · CONSTRUCTION
Build and test in Bolts.
`planning/tasks.md`

4 · TRANSITION
Release and document.
`CHANGELOG` + tag

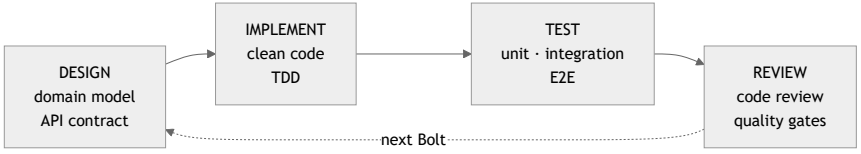
5 · PRODUCTION
Operate and improve.
runbook · dashboards

6 · RETIREMENT
Decommission cleanly.
decommission plan

PHASES ARE GATED, NOT SUGGESTED

- Every phase declares **entry and exit criteria**. DISCOVERY cannot exit without approved specs; CONSTRUCTION cannot start without a task breakdown.
- A **handoff matrix** makes illegal transitions explicit: plan → implement is refused, because it skips task breakdown.

INSIDE CONSTRUCTION: THE BOLT



Each Bolt runs on its own branch and must clear the full quality gate before it can be called done.

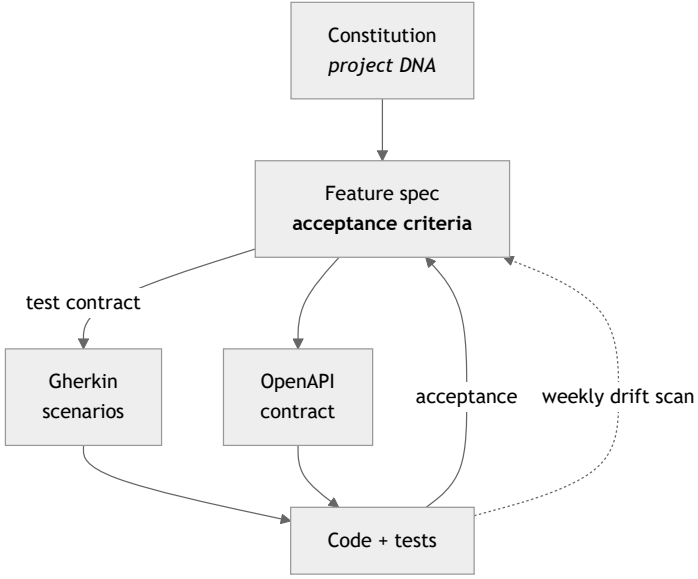
The gate is only as fast as the pipeline that runs it. At this cadence **CI/CD stops being a nicety** — where a client's pipeline cannot validate an increment in hours, the Bolts stay and the Sprint around them gets very short.

© 2026 Avanade Inc. All rights reserved.

Specs before code — enforced, not encouraged

The **acceptance criteria** are the contract — they drive the tests, the code and the sign-off

THE TRACEABILITY SPINE



FIVE MECHANISMS THAT MAKE IT STICK

- **Acceptance criteria are the test contract.** Every spec carries them, classified `@smoke`; they become the Gherkin the build is verified against, and the basis on which the feature is accepted.
- **Phase gates.** No spec, no construction. Exit criteria are checked, not assumed.
- **Every feature has a ticket.** Creating it is mandatory — GitHub Issue by default, Jira or Azure Boards where the client works there — and its id is recorded in the spec, so spec, branch, commits and PR resolve to one thread.
- **Drift analysis.** A dedicated agent cross-checks every spec entity to its data model, DTO, endpoint and test — and flags orphan code.
- **CI enforcement.** A scheduled workflow scores spec-to-code drift **0–10** every week and opens a labelled issue when it finds any.

The framework's own words: **"Specs Before Code"** and **"Constitution is law — never violate its constraints."**

© 2026 Avanade Inc. All rights reserved.

Advantages across the SDLC

What the framework contributes at each phase — and the gate that closes it

WHAT BOLT ADDS, PHASE BY PHASE

Phase	What you get that you didn't have
Inception	A ratified constitution: stack, architecture, quality and security decided **once**, explicitly, machine-readable
Discovery	Specs, use cases, Gherkin and OpenAPI contracts as a **connected set**, not separate documents
Construction	2–3 day Bolts on their own branches, TDD red/green/refactor agents, and a gate that blocks merge
Transition	Semantic-version releases with changelog, deployment guide and release notes from templates
Production	Runbooks, observability config, postmortems and improvement proposals as artifacts
Retirement	A decommission plan with impact analysis, data handling and sign-off — the phase most methods omit
Cross-phase	Status and alignment reports, security and dependency analysis, docs — plus an ADR whenever a decision is taken, in any phase

THE GATE EVERY BOLT MUST CLEAR

Check	Threshold
Linting	0 errors
Unit tests	100% pass
Line coverage	≥ 80%
Branch coverage	≥ 75%
Mutation score	≥ 70%
Architecture tests	100% pass
Code review	Approved
Constitution compliance	Pass

Modernization adds one more

An **equivalence gate**: ≥ 95% parity against the legacy system and **100%** of priority-zero behaviour characterised by tests — before anything is switched over.

Enforced thresholds, not measured outcomes — the framework calls this a "non-optional checkpoint: no PR merge without passing gates".

© 2026 Avanade Inc. All rights reserved.

Where it applies

Five documented entry points — greenfield is only one of them

Greenfield

An idea or a blank repository. Constitution first, then specs, then Bolts.

Brownfield

COBOL, VB6, legacy .NET or Java. Analyse, characterise, then modernise behind an equivalence gate.

From a requirement

An RFP or tender. Extract the requirements, then produce architecture, plan and estimate — before anyone commits.

Discovery only

Understand and document a domain or system. Models, specs and decision records — no code. The gate is human: **a named expert signs it off**.

Single feature

An existing Bolt project. Add one feature through the full spec-to-review chain.

ONE FRAMEWORK, CONFIGURED PER ENGAGEMENT

A **practice** — Apps & Infra, Data & AI, CRM or Custom — activates only the areas the work needs, each with its own slice of the governing document, skills and tooling. Nothing is forked per project. Highlighted: Apps & Infra.

backend · frontend · cloud-platform · work-management · common · data · ai · integration

AND IT ADAPTS TO THE SHAPE OF THE WORK

The framework detects the shape of the work — back end only, front end only, infrastructure only, or full stack — and changes what it produces: an infrastructure project is never asked for a domain model or an API contract.

Backend stack templates ship for **.NET, Node.js/NestJS, Python/FastAPI and Go**.

avanade

© 2026 Avanade Inc. All rights reserved.

Do what matters

Custom development

When code stops being the constraint, what the client is buying changes

Code is no longer the constraint — what the client buys is **definition and verification**, validated **as we go** rather than at the end.



Turnkey — fixed scope

- Needs a **detailed definition phase** up front: features, use cases and acceptance criteria — each one becoming an executable test.
- The spec *is* the contract, and drift from it is **scored weekly** — not discovered at UAT.
- Effort, cost and a go/no-go scorecard come from the RFP path.
- **Watch:** a spec error, once signed off, becomes contractual scope.



Agile — evolving scope

- Course changes happen **between Bolts, not inside one** — two to three days is the smallest unit of committed work, so changing direction costs days.
- Re-prioritise freely: the constitution and the quality gates do not move, so speed never comes out of quality.
- Right while requirements are still moving, or the domain is being discovered with the client.

What decides: requirement stability, and who carries scope risk. Spec-driven delivery makes fixed price **more governable than usual** — scope is written as tests before anyone commits — but only if the definition phase is funded.

avanade

© 2026 Avanade Inc. All rights reserved.

One-pager — Customers

What changes in how your software gets built, and what you can hold us to

WHAT CHANGES

- **The specification becomes the contract** — written, agreed and versioned before code exists, not reconstructed afterwards.
- **Decisions are made once, in the open.** Your stack, architecture, quality and security rules are ratified up front in a document you sign off.
- **Work arrives every few days** as tested, reviewed increments, not one delivery after a long silence. How fast depends on your pipeline: without automated build and release we start with short cycles and tighten them as it matures.
- **AI is used under supervision**, inside rules you approved, with a record of what it did.
- **Drift is detected, not discovered.** A weekly check reports where the code has diverged from what was agreed.

WHAT YOU CAN HOLD US TO

- **Agents must obey the rules you signed.** They must read the governing document, validate against it, **fail** rather than proceed on a violation, and log every check for audit.
- **Every architectural decision is recorded** — dated, with the alternatives considered.
- **Quality is numeric**, not editorial: coverage, mutation and architecture thresholds block a merge.
- **Security and compliance are in scope by default** — OWASP Top 10, plus GDPR, HIPAA, SOC 2 and PCI-DSS checklists.
- **Humans approve what matters.** Production deploys need manual approval; modernization stops at five checkpoints.

WHAT WE NEED FROM YOU

- **Stakeholder time early.** The inception and discovery phases need decisions from people who can make them.
- **Sign-off on the governing document.** It is the reference every agent and engineer works against.
- **A named owner for exceptions.** Amendments require review and approval — deliberately.

No lock-in

Specifications, decisions, plans and tests are plain Markdown and standard formats in **your** repository. If we stop working together tomorrow, everything that describes your system stays with you — and so does **the way of working**: your team can keep running the same method.

avanade

© 2026 Avanade Inc. All rights reserved.

Do what matters

What efficiency to expect

An analyst benchmark, segmented — an expectation, not a commitment

25–30% **across the lifecycle** — the analyst benchmark for applying AI beyond coding, against the **~10%** typically seen where it is used for code alone. Bolt Framework is **built to reach it**.

WHERE THE GAIN CONCENTRATES

Work	Realistic band
Greenfield, well-specified features	Upper end — no figure published until verified
Rework and verification overhead	Material — this is Bolt's mechanism
Brownfield, high-complexity legacy	0–15%

Saying the third row out loud is what buys the first: a single blended number swings by **4×** between the top row and the bottom one. Where we cannot cite a figure, **we do not print one**.

WHAT THE BAND ASSUMES

- A working **CI/CD harness**, automated tests with a usable coverage baseline, version-control discipline and small batches.
- AI-accessible internal documentation, and an **agreed baseline** taken before we start.
- Where these are missing, the first Bolts build them — the gain then lands **later and lower**. Expect a **J-curve** before the curve.
- Measured by **first-pass acceptance rate**, never by coding throughput.

Gartner, *How to Capture AI-Driven Productivity Gains Across the SDLC* (11 Apr 2025) — 25–30% by 2028 across the SDLC vs ~10% code-focused. Segmentation: **Stanford** (Denisov-Blanch) and **McKinsey**. J-curve: **DORA** ROI report 2026.01. Measurement: **Thoughtworks Technology Radar** Vol. 34, which places coding throughput in *Caution*. **Indicative expectation, never contractual** — Bolt Framework publishes no efficiency measurement of its own.

© 2026 Avanade Inc. All rights reserved.

An operating model, not a tool

It transfers with the work — the specifications, decision records and tests stay in **your** repository, readable without us. **Next step:** a short assessment to map your delivery baseline and agree where the first Bolts land.

Higher education — VB6 migration

The engagement Bolt Framework came out of

APPENDIX · 1 OF 3 **DELIVERED · IN MAINTENANCE**

DISCOVERY CONSTRUCTION PRODUCTION

PROBLEM

A VB6 enrolment system had to be replaced. The functional knowledge that defined it lived with its users rather than in any documentation — so the specification did not exist to be migrated.

SOLUTION

Bolt's legacy agents read the source, the documentation and the database to recover the schema and the business rules, written as Given/When/Then — which became the tests the rebuild was checked against. The rebuild then ran as governed Bolts: .NET 8 microservices over Oracle, Clean Architecture with domain events, and an Angular 16 front end on Atomic Design. The same process now runs corrective and evolutive maintenance.

> **The functional knowledge exists in writing again** — the system no longer depends on who is still in the building. It runs on Azure, scaling up for enrolment peaks and back down in the off-season.



What it cost: part of the framework was being built at the same time as the project was delivered. Client anonymised.

© 2026 Avanade Inc. All rights reserved.

Automotive — support agent

Constitution to production, on its own stack and toolchain

APPENDIX · 2 OF 3 **IN PRODUCTION**

INCEPTION → PRODUCTION full lifecycle

PROBLEM

A vast, highly heterogeneous dealership network generated more support tickets than the team could absorb. Lead times stretched badly, and reopened and reassigned tickets were a large part of the cost.

SOLUTION

An end-user agent that guides people as they raise ServiceNow requests, so tickets arrive correctly formed and stop bouncing between support teams. Built greenfield under Bolt from the constitution onwards: microservices in Python and Node.js with a React front end, **Azure AI Foundry** as the agent platform, a **ServiceNow** integration, and Azure DevOps work-item sync. Bolts ran inside a three-week coordination cycle.

From zero definition to production in four months, with a team of four.



What it cost: response-time commitments force the team to balance project work against support load. Client anonymised; the four-month figure is a delivery fact about one engagement, not a benchmark.

© 2026 Avanade Inc. All rights reserved.

Banking — a client-owned SDLC

In flight — the framework as someone else's standard

APPENDIX · 3 OF 3 **UNDERWAY**

INCEPTION — enterprise-wide

PROBLEM

Development is spread across internal teams and a supplier ecosystem with no common process. The bank wants one AI-supported SDLC that applies to both — and wants to **own it**, not rent it.

SOLUTION

Bolt Framework as the base for that process, adapted to the bank's own tools, requirements and controls, on GitHub end to end. Delivered jointly with Accenture, starting from the client's VDI setup and continuing into the definition of the AI-supported SDLC flow itself.

> The target is **the bank's own**: efficiency in time-to-market and cost, assessed at one and two years. That is the client's objective, not a Bolt result.



What it cost: every step is audited and reviewed with security. The flow is agreed and clear, but progress moves at the pace review allows. Client anonymised; cleared for anonymous reference.

© 2026 Avanade Inc. All rights reserved.